CG UWR Technicals Package 2.2
README

Jakub Kowalski, Marek Szykuła

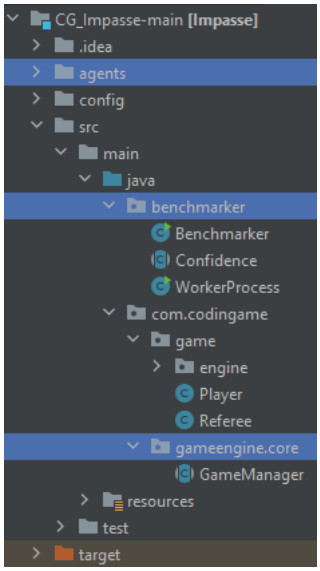
December 13, 2023

The package is confidential.
Distribution outside of people affiliated with **University of Wrocław** is strictly prohibited.

1 Preparing Game Referee

Step 1. Download Referee
Usually, the source code of a game referee (engine) is available via a GitHub repository linked in the game rules. Referee source is a Java project with a set layout, according to the CodinGame game-developing requirements.
Just download and unpack a `zip` archive or clone the repository.

Step 2. Copy Package Content
Copy the content of the `CG technicals package` into the main folder of the repository.
The folder structure should look as follows (directories updated with the package are highlighted):



Step 3. Fixing the Referee
With parallel computations, your agents will timeout even if they run OK on CodinGame. This fix is required but assumes that your agent programs **has some internal time tracking and does not exceed normal CG times**. Usually in referee `src/main/java/com/codingame/game/Referee.java` there is something like

```
gameManager.setFirstTurnMaxTime(TIMELIMIT_INIT);
gameManager.setTurnMaxTime(TIMELIMIT_TURN);
```

You can increase the limits freely, e.g., $\times 100$.

Step 3a. (optional) Fixing Non-determinism
We need to **determinize code**. If data structures such as `HashSet / HashMap` are used, we have to substitute them with `LinkedHashSet / LinkedHashMap`.

Step 3b. (optional) Fixing Static Structures
If a game **modifies its static variables** during the play, we have to reset them before each game.

Step 3c. (optional) Fixing League Level
For games with **multiple leagues with rule changes**, it is important that our agent plays the rules variant it is made for.
It can be easily set within `runOnePlay` method of `benchmarker.WorkerProcess.java`.

2 Runing Game Referee and Agents

Requirements:
`jdk >= 11`
`maven`

Agents Setup:
Your agents' executables should be placed in the `agents` directory in the referee.

2.1 In IDE

The recommended approach is to use **IntelliJ IDEA** and just open referee project / new from existing sources.
You can change benchmark parameters within `benchmarker/Benchmark.java` file and run computations directly from IDE:

```
static int NUM_THREADS = 4;
static String AGENTS_PATH = "./agents/";
static String AGENT1 = "v26";
static String AGENT2 = "v21";
static final long BASE_SEED = 1;
static long NUM_PLAYS = 100;
```

And then just `Run 'Benchmarker.main()'`.

2.2 In terminal

Run `rebuild.sh` script to build referee `jar` executable (once).
Modify `run.sh` script substituting `GAMENAME` with the name of the game, which can be found in `<artifactId>` tag inside `pom.xml` file.

To run experiments, simply run `run.sh` providing the arguments you want (not provided values use default from `Benchmark.java`):
First two arguments are for your agents' executable names.
Third argument is for the number of games to run.
Fourth argument is for the number of threads to use.

3 Remarks

3.1 Non-executable agents

To run agents which are not in a form of executable files, e.g., `python` programs, you have to modify the `Benchmark.java` so that the result of combining variable `AGENTS_PATH` with `AGENT1`, `AGENT2` or both will have a form `python3 agents/main.py`.

3.2 Operating System

It is strongly advised to perform setup and computations on some Unix-family system.
For Windows users, I recommend taking advantage of Windows Subsystem for Linux (**WSL**). It works really nicely.

4 Other Tools

4.1 CGStats

See http://cgstats.magusgeek.com/app

- A webpage containing cumulated stats from different opponents' submissions
- Useful to discover problematic opponents
- The stats disappear over time

4.2 CGBenchmark

See https://github.com/s-vivien/CGBenchmark

- Tool for testing your agent against real opponents
- Setup: cookie, profile link, code IDs
- Current CG play limits:
  - 10 plays immediately on fresh (cooldown 0)
  - 80 plays (cooldown 45 $\rightarrow$ 1h; cooldown 0 $\rightarrow$ $>$ 30 min)
  - 25 plays / h on average, 600 plays in 24h (cooldown 144)
- Useful for detecting timeouts and errors
- Package contains fixed skipping on errors

4.3 CGButtons

See https://github.com/DomiKoPL/CGButtons/tree/stdin-button

- Chrome extension for fast copying of agent's stderr
- Lines prefixed with `?>` can be filtered and used for e.g., local input.